

如何解决LLM致幻— Gemini

这是一个非常经典的“冷启动”（Cold Start）问题。你的系统在面对未知或新股票时效果差，是因为现有的防致幻体系高度依赖内部向量数据库（Vector DB）或本地文档库。当新股票没有被索引时，Agent 要么因为检索不到内容而拒答，要么就开始利用大模型的 parametric memory（参数记忆）生搬硬套，导致致幻。

要解决第一次分析（文档里没有相关信息）的股票，你需要引入一套动态热加载与实时外挂知识库（Just-In-Time Ingestion & Live Search）的机制。

以下是具体的优化架构方案：

核心解决方案：动态热加载与三层防御机制

你可以把防止致幻的体系升级为“动态补充 → 严格对齐 → 交叉验证”的闭环。

1. 触发“冷启动”动态数据编排 (Dynamic Ingestion Pipeline)

当用户输入一个新股票代码/名称时，系统应该先通过一个路由器 Agent (Router Agent) 进行检索前置检查：

- **步骤 A（检测）**：检索本地文档库，若相关度 Score 低于阈值，触发“冷启动”流。
- **步骤 B（实时抓取）**：调用搜索引擎 API（如 Perplexing, Tavily, Google Search）或金融数据接口（如 Yahoo Finance, Alpha Vantage），定向抓取该股票的**最新财报摘要、近期新闻、行业报告**。
- **步骤 C（内存热加载）**：抓取到的文本**不急于永久持久化**，而是直接在内存中建立一个临时向量索引（Ephemeral Vector Index），或者直接放入该会话的长上下文（Long Context Window）中，作为本次分析的强约束边界。

2. 引入两阶段提示词工程 (2-Stage Prompting)

为了防止没有文档时 Agent 瞎编，必须严格限制其推理边界。

- **阶段一：事实提取 Agent (Fact Extractor)**
 - 只允许它从刚刚实时抓取的内容（或缺失提示）中提取关键数据（如 P/E, 营收增长率, 风险提示），**绝对禁止任何推论**。

- Prompt 显式声明：“如果实时抓取的信息中没有包含 XX 指标，直接返回 [数据缺失]，绝对不要根据你的常识进行猜测。”
- **阶段二：严格对齐分析 Agent (Grounded Analyst Agent)**
 - 基于阶段一提取的“干净事实”进行分析。
 - 要求模型在输出的每一句关键结论后，强制加上 [Source: Live Search / Document X]。

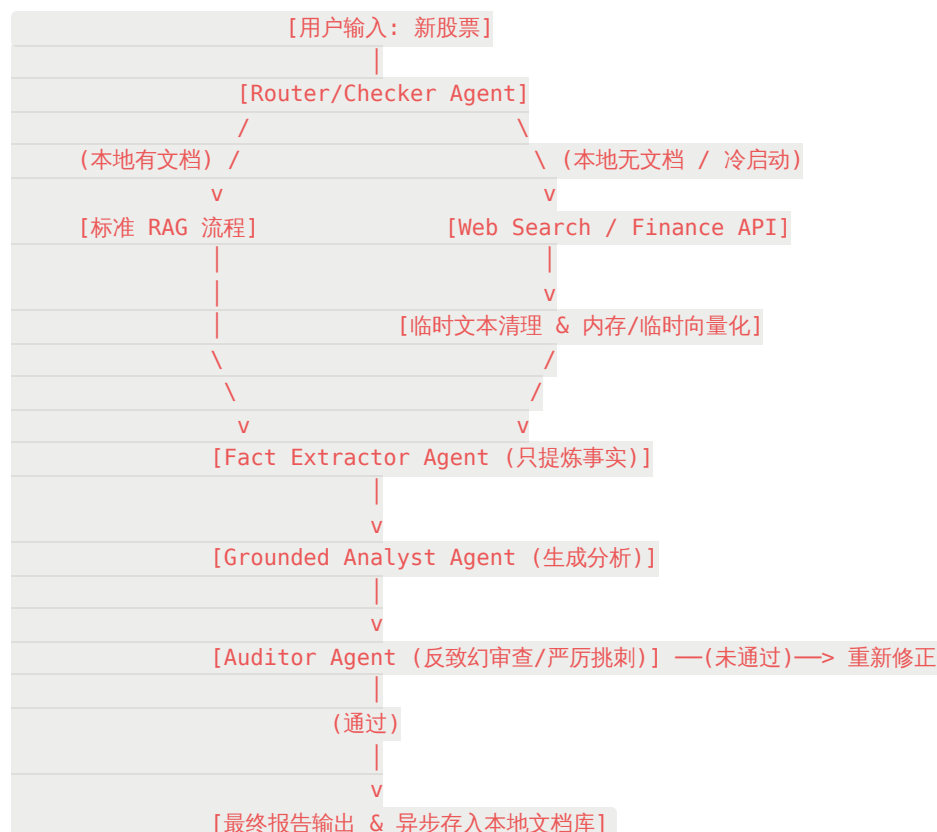
3. 多 Agent 辩论/交叉验证 (Multi-Agent Debate)

既然是多 Agent 系统，可以利用角色对抗来压制致幻：

- **生成 Agent (Analyst)：**负责根据实时抓取的数据生成初稿。
- **挑刺 Agent (Critic/Auditor)：**负责审查。它的唯一任务是拿着 Analyst 的初稿，和实时抓取的 Raw Data 进行对齐。如果发现初稿中某句话在 Raw Data 里找不到支撑，直接打回重写。

优化后的 Agent workflow 架构

你可以用你的 LangGraph 架构，把流程调整为如下并行/串行结合的拓扑结构：



落地实施的具体建议

1. **设置“诚实度”惩罚（Honesty Penalty）：** 在 System Prompt 中加入极端 case 惩罚。例如：“如果因为是新股票导致信息不足，给出 60 分的保守客观分析，并列出具『未知风险/缺失数据清单』，比勉强给出 100 分但含有幻觉的分析，得分要高得多。”
2. **异步落库（Async Ingestion）：**
第一次分析完后，把刚刚抓取的所有清洗后的数据，**异步投递到你的 Celery 队列或者 LangGraph 的后台节点**，完成切片（Chunking）和向量化，存入你正式的文档库。这样，第二次分析时它就变成“老股票”了。
3. **区分 Parametric vs. Non-parametric Memory：**
对于新股票，在调用大模型时，将 `temperature` 调得更低（例如 `0.0` 或 `0.1`），逼迫模型进行强确定性输出，断绝其发挥想象力的空间。

你目前的防致幻体系是用什么规则做的？（比如是用的 LangChain/LangGraph Self-RAG 架构，还是单纯靠 Prompt 约束？）我们可以针对你现有的核心代码节点做针对性的接入。